



TECNOLÓGICO NACIONAL DE
MÉXICO

INSTITUTO TECNOLÓGICO DEL VALLE DE
OAXACA



ADMINISTRACIÓN Y ORGANIZACIÓN DE DATOS

“A3.3. DESARROLLAR UN PROYECTO PARA
CREAR Y LEER UN ARCHIVO XML”

CARRERA:

INGENIERÍA INFORMÁTICA

GRUPO:

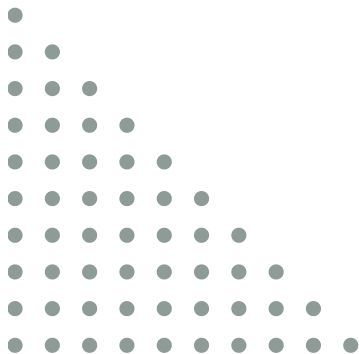
4-A

ELABORÓ:

- ERICK ALEJANDRO MARTÍNEZ
GARCÍA

DOCENTE:

- BENEDICTO RAMÍREZ SANTIAGO



Introducción

El manejo de archivos estructurados es una competencia fundamental en el desarrollo de software. Entre los formatos más utilizados para el almacenamiento e intercambio de información se encuentra XML (eXtensible Markup Language), un lenguaje de marcado que permite representar datos jerárquicos de forma legible tanto para humanos como para máquinas. A diferencia del formato CSV, XML permite definir estructuras de datos más complejas y validar su contenido mediante esquemas.

En el presente proyecto se desarrolló una aplicación en Java orientada a objetos utilizando el patrón de diseño Modelo-Vista-Controlador (MVC). El sistema permite registrar y consultar huéspedes almacenando la información en un archivo XML llamado `huespedes.xml`. Los datos capturados incluyen el identificador, nombre, apellidos y procedencia de cada huésped.

Para la implementación se utilizaron las APIs de Java para manipulación de documentos XML: `DocumentBuilder`, `DocumentBuilderFactory`, `Document`, `Element` y `NodeList` del paquete `org.w3c.dom`, así como `Transformer` y `TransformerFactory` para la escritura del archivo. La interfaz gráfica fue desarrollada con Java Swing, proporcionando una ventana única con campos de captura y un área de texto para la visualización de los registros.

El objetivo principal de esta práctica es comprender el funcionamiento de los archivos XML en Java, aplicar los conceptos de programación orientada a objetos y reforzar la implementación del patrón MVC en el desarrollo de aplicaciones con persistencia de datos en archivos estructurados.

Desarrollo

Modelo

ArchivoXML.java

La clase ArchivoXML constituye el núcleo del manejo del archivo de persistencia. En su constructor verifica si el archivo huespedes.xml existe; en caso de no existir, crea un nuevo documento XML con un elemento raíz llamado huespedes y lo guarda en disco mediante el método guardarDocumento(). Esta clase expone dos métodos públicos: guardar(Huesped h), que agrega un nuevo nodo huesped con sus subelementos id, nombre, apellidos y procedencia al documento existente; y leer(), que recorre todos los nodos huesped del archivo y construye una cadena de texto con la información de cada registro.

Los métodos privados obtenerDocumento() y guardarDocumento(Document doc) encapsulan respectivamente la carga del archivo XML hacia un objeto Document y la escritura del objeto Document de regreso al archivo, utilizando Transformer para la serialización.



```
1 package modelo;
2
3 import java.io.File;
4 import javax.xml.parsers.DocumentBuilder;
5 import javax.xml.parsers.DocumentBuilderFactory;
6
7 import javax.xml.transform.Transformer;
8 import javax.xml.transform.TransformerFactory;
9 import javax.xml.transform.dom.DOMSource;
10 import javax.xml.transform.stream.StreamResult;
11
12 import org.w3c.dom.*;
13
14 public class ArchivoXML {
15
16     private final String archivo = "huespedes.xml";
17
18     public ArchivoXML() {
19
20         try {
21
22             File f = new File(archivo);
23
24             if (!f.exists()) {
25
26                 DocumentBuilderFactory factory =
27                     DocumentBuilderFactory.newInstance();
28
29                 DocumentBuilder builder =
30                     factory.newDocumentBuilder();
31
32                 Document doc =
33                     builder.newDocument();
34
35                 Element raiz =
36                     doc.createElement("huespedes");
37
38                 doc.appendChild(raiz);
39
40                 guardarDocumento(doc);
41             }
42
43         } catch (Exception e) {
44
45             e.printStackTrace();
46         }
47     }
48 }
```

Figura 1. ArchivoXML.java — Parte 1: imports, atributo archivo, constructor y creación del documento inicial.

```

49 public void guardar(Huesped h) {
50
51     try {
52
53         Document doc = obtenerDocumento();
54
55         Element raiz =
56             doc.getDocumentElement();
57
58         Element huesped =
59             doc.createElement("huesped");
60
61         Element id =
62             doc.createElement("id");
63
64         id.appendChild(
65             doc.createTextNode(
66                 String.valueOf(h.getId())));
67
68         Element nombre =
69             doc.createElement("nombre");
70
71         nombre.appendChild(
72             doc.createTextNode(
73                 h.getNombre()));
74
75         Element apellidos =
76             doc.createElement("apellidos");
77
78         apellidos.appendChild(
79             doc.createTextNode(
80                 h.getApellidos()));
81
82         Element procedencia =
83             doc.createElement("procedencia");
84
85         procedencia.appendChild(
86             doc.createTextNode(
87                 h.getProcedencia()));
88
89         huesped.appendChild(id);
90         huesped.appendChild(nombre);
91         huesped.appendChild(apellidos);
92         huesped.appendChild(procedencia);
93
94         raiz.appendChild(huesped);
95
96         guardarDocumento(doc);

```

Figura 2. ArchivoXML.java — Parte 2: método guardar(), construcción de nodos XML por huésped.

```

98     } catch (Exception e) {
99
100         e.printStackTrace();
101     }
102 }
103
104 public String leer() {
105
106     StringBuilder datos =
107         new StringBuilder();
108
109     try {
110
111         Document doc =
112             obtenerDocumento();
113
114         NodeList lista =
115             doc.getElementsByTagName("huesped");
116
117         for (int i = 0; i < lista.getLength(); i++) {
118
119             Element h =
120                 (Element) lista.item(i);
121
122             datos.append("ID: ")
123                 .append(h.getElementsByTagName("id")
124                     .item(0).getTextContent())
125                 .append("\n");
126
127             datos.append("Nombre: ")
128                 .append(h.getElementsByTagName("nombre")
129                     .item(0).getTextContent())
130                 .append("\n");
131
132             datos.append("Apellidos: ")
133                 .append(h.getElementsByTagName("apellidos")
134                     .item(0).getTextContent())
135                 .append("\n");
136
137             datos.append("Procedencia: ")
138                 .append(h.getElementsByTagName("procedencia")
139                     .item(0).getTextContent())
140                 .append("\n");
141
142             datos.append("-----\n");
143         }
144     }

```

Figura 3. ArchivoXML.java — Parte 3: método leer(), recorrido de NodeList y construcción del texto de salida.

```

145 } catch (Exception e) {
146     e.printStackTrace();
147 }
148
149
150 return datos.toString();
151 }
152
153 private Document obtenerDocumento()
154     throws Exception {
155
156     DocumentBuilderFactory factory =
157         DocumentBuilderFactory.newInstance();
158
159     DocumentBuilder builder =
160         factory.newDocumentBuilder();
161
162     return builder.parse(
163         new File(archivo));
164 }
165
166 private void guardarDocumento(Document doc)
167     throws Exception {
168
169     Transformer transformer =
170         TransformerFactory.newInstance()
171             .newTransformer();
172
173     DOMSource source =
174         new DOMSource(doc);
175
176     StreamResult result =
177         new StreamResult(
178             new File(archivo));
179
180     transformer.transform(source, result);
181 }
182

```

Figura 4. ArchivoXML.java — Parte 4: métodos privados obtenerDocumento() y guardarDocumento().

Huesped.java

La clase Huesped representa la entidad del dominio. Contiene cuatro atributos privados: id (entero), nombre, apellidos y procedencia (cadenas de texto). Cuenta con un constructor parametrizado que inicializa todos los atributos y cuatro métodos getter: getId(), getNombre(), getApellidos() y getProcedencia(), que permiten acceder a los datos del objeto desde otras clases del sistema.

```
1  package modelo;
2
3  public class Huesped {
4
5      private int id;
6      private String nombre;
7      private String apellidos;
8      private String procedencia;
9
10     public Huesped(int id,
11                     String nombre,
12                     String apellidos,
13                     String procedencia) {
14
15         this.id = id;
16         this.nombre = nombre;
17         this.apellidos = apellidos;
18         this.procedencia = procedencia;
19     }
20
21     public int getId() {
22         return id;
23     }
24
25     public String getNombre() {
26         return nombre;
27     }
28
29     public String getApellidos() {
30         return apellidos;
31     }
32
33     public String getProcedencia() {
34         return procedencia;
35     }
36 }
```

Figura 5. Huesped.java — Atributos, constructor parametrizado y métodos getter.

Controlador

HuespedControlador.java

El controlador HuespedControlador actúa como intermediario entre la vista y el modelo. Mantiene una instancia de ArchivoXML como atributo privado, la cual es inicializada en el constructor. Expone dos métodos públicos: guardar(Huesped h), que delega la operación de persistencia al objeto ArchivoXML; y listar(), que solicita al modelo la cadena con todos los registros y la retorna a la vista para su presentación.



```
1 package controlador;
2
3 import modelo.ArchivoXML;
4 import modelo.Huesped;
5
6 public class HuespedControlador {
7
8     private ArchivoXML archivo;
9
10    public HuespedControlador() {
11
12        archivo = new ArchivoXML();
13    }
14
15    public void guardar(Huesped h) {
16
17        archivo.guardar(h);
18    }
19
20    public String listar() {
21
22        return archivo.leer();
23    }
24 }
```

Figura 6. HuespedControlador.java — Constructor, método guardar() y método listar().

Capturas de Ejecución

Entrada de Datos

Al ejecutar la aplicación se presenta la ventana Registro de Huéspedes, que contiene cuatro campos de texto para capturar el ID, nombre, apellidos y procedencia del huésped, así como dos botones: Guardar, para registrar los datos en el archivo XML, y Leer, para consultar los registros existentes. El área de visualización ubicada a la derecha permanece en blanco hasta que se presiona el botón Leer.

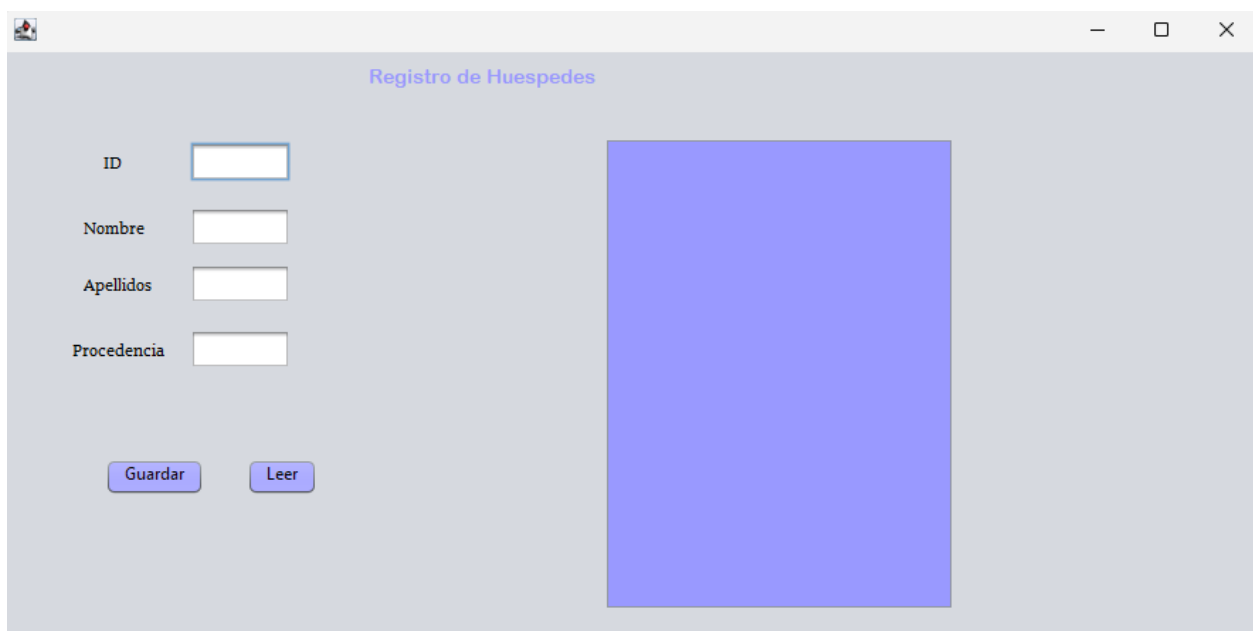


Figura 7. Vista principal de la aplicación sin registros consultados.

Salida de Datos

Al presionar el botón Leer, el sistema llama al controlador, que a su vez invoca el método leer() de ArchivoXML. Este recorre todos los nodos del archivo XML y construye una cadena de texto con la información de cada huésped, la cual se despliega en el área de texto de la interfaz. Cada registro muestra el ID, nombre, apellidos y procedencia, separados por una línea divisoria.



Figura 8. Vista principal mostrando los huéspedes consultados desde el archivo XML.

Conclusión

La realización de este proyecto permitió comprender el uso del formato XML como mecanismo de almacenamiento de datos estructurados en aplicaciones Java. A diferencia del formato CSV, XML ofrece una representación jerárquica que facilita la organización de información con múltiples atributos por registro, como en el caso del sistema de registro de huéspedes desarrollado.

Durante el desarrollo se aplicaron los conceptos del patrón Modelo-Vista-Controlador, logrando una separación clara de responsabilidades: la clase ArchivoXML encapsula toda la lógica de lectura y escritura del archivo; la clase Huesped representa la entidad del dominio; el controlador HuespedControlador coordina las operaciones entre ambas capas; y la vista gestiona la interacción con el usuario mediante Java Swing.

Se reforzaron las habilidades de manipulación de documentos XML utilizando la API DOM de Java, incluyendo la creación de documentos, la adición de elementos y atributos, y la serialización del árbol de nodos hacia un archivo físico. Estas técnicas son ampliamente utilizadas en el desarrollo de software para la integración de sistemas y el intercambio de datos entre aplicaciones.

En conclusión, el proyecto contribuyó de manera significativa al aprendizaje del manejo de archivos XML en Java, consolidando competencias en programación orientada a objetos, diseño con patrones arquitectónicos y desarrollo de interfaces gráficas, habilidades esenciales en la formación de un ingeniero en informática.